МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТІ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Кафедра систем штучного інтелекту

Лабораторна робота №3

з дисципліни
«Операційні системи»

Виконав:
студент групи КН-114
Добрій Назарій

Викладач:
Кривенчук Ю.П.

Львів – 2020 р.

Тема. Виконання задачі в декількох потоках в ОС Windows.

Мета. Навчитись реалізовувати розпаралелювання алгоритмів за допомогою багатопоточності в ОС Windows з використанням функцій WinAPI.

Завдання.

1. Реалізувати заданий алгоритм в окремому потоці.
2. Виконати розпаралелювання заданого алгоритму на 2, 4, 8 потоків.
3. Реалізувати можливість зміни пріоритету певного потоку.
4. Результати виконання роботи відобразити у звіті.

Варіант 6

6. Створити масив N елементів і відсортувати його елементи у порядку ***спадання*** за допомогою методу ***«вибірки»***. Елементи масиву згенерувати випадковим чином за допомогою вбудованих функцій.

Код реалізації програми:

```cpp
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <windows.h>
#include <vector>
#include <ctime>

using namespace std;

HANDLE hThreads[8];

DWORD WINAPI selectionSort(void* param) {
    vector<int>& tmp = *(vector<int>*) param;
    int i, j, min;
    for (i = 0; i < tmp.size() - 1; i++) {
        min = i;
        for (j = i + 1; j < tmp.size(); j++)
            if (tmp[j] < tmp[min])
                min = j;
        swap(tmp[i], tmp[min]);
    }
    return 0;
}

vector<int> complete(vector<vector<int>> arr, int n, int k) {
    vector<int> completed(n);
    for (int i = 0; i < n; ++i) {
        int min = 1000000;
        int th = -1;
        for (int j = 0; j < k; j++) {
            if (!arr[j].empty() && arr[j][0] < min) {
                min = arr[j][0];
                th = j;
            }
        }
        completed[n - i - 1] = min;
        arr[th].erase(arr[th].begin());
    }
```

```cpp

int main() {

    srand(time(NULL));

    int elnum, thnum;
    vector<vector<int>> arr;

    cout << "Enter the number of elements: ";
    cin >> elnum;
    vector<int> input(elnum);
    for (int i = 0; i < elnum; ++i) {
        input[i] = rand() % elnum + 1;
        cout << input[i] << ' ';
    }
    cout << endl;

    cout << "Enter the number of threads: ";
    cin >> thnum;
    for (int i = 0; i < thnum; ++i) {
        vector<int> tmp;
        for (int j = i * elnum / thnum; j < (i + 1) * elnum / thnum; j++) {
            tmp.push_back(input[j]);
        }
        arr.push_back(tmp);
    }

    for (int i = 0; i < thnum; ++i) {
        int ThreadPriority;

        DWORD ThreadID;

        HANDLE thread = CreateThread(NULL, 0, selectionSort, &arr[i], 0, &ThreadID);
        hThreads[i] = thread;

        if (hThreads[i] == NULL)
        {
        {
            cout << "Thread Error" << endl;
            ExitProcess(1);
        }

        //Opend Thread
        HANDLE hOpenThread = OpenThread(
            THREAD_SET_LIMITED_INFORMATION,
            FALSE,
            ThreadID
        );

        if (NULL == hOpenThread)
        {
            cout << "Open Thread Function Failed & Error - " << GetLastError() << endl;
            ExitProcess(2);
        }

        // Set Thread Priority
        int mode;
        cout << "Set priority: " << endl;
        cout << "Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = ";
        cin >> mode;

        while (mode > 7 || mode < 0)
        {
            cin >> mode;
        }

        switch (mode) {
        case 1:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_LOWEST);
            break;
        case 2:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_BELOW_NORMAL);
            break;
        case 3:
```

```
                SetThreadPriority(hThreads[i], THREAD_PRIORITY_NORMAL);
                break;
            case 4:
                SetThreadPriority(hThreads[i], THREAD_PRIORITY_ABOVE_NORMAL);
                break;
            case 5:
                SetThreadPriority(hThreads[i], THREAD_PRIORITY_HIGHEST);
                break;
            case 6:
                SetThreadPriority(hThreads[i], THREAD_PRIORITY_TIME_CRITICAL);
                break;
            }

            //Delay
            //Sleep(2000);

            //Getting Thread Priority
            ThreadPriority = GetThreadPriority(thread);

            //WaitForSingleObject for Thread
            WaitForSingleObject(hOpenThread, INFINITE);

            cout << "Thread Priority = " << ThreadPriority << endl;

            //Closing Handle of Thread
            CloseHandle(hOpenThread);
        }

        vector<int> completed = complete(arr, elnum, thnum);
        for (int i = 0; i < elnum; ++i) {
            cout << completed[i] << ' ';
        }
        cout << endl;

        system("pause");
        return 0;
    }
```

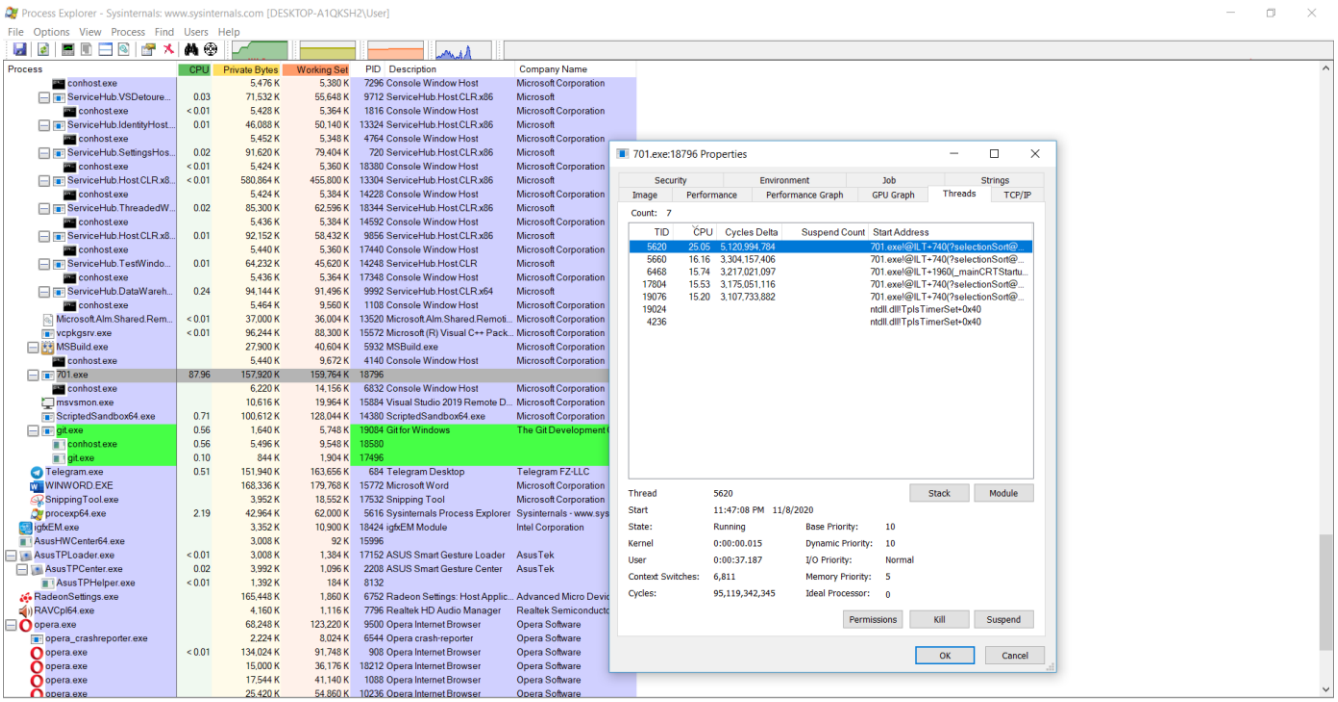
Результат виконання програми у консолі:

```
C:\Users\User\source\repos\701\Debug\701.exe

Enter the number of elements: 1000000

Enter the number of threads: 2
Set priority:
Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = 5
Thread Priority = 2
Set priority:
Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = 3
Thread Priority = 0
Press any key to continue . . .
```

Результат виконання програми у process explorer:



Висновок: я навчився реалізовувати розпаралелювання алгоритмів за допомогою багатопоточності в ОС Windows з використанням функцій WinAPI. Також змінювати пріорітет заданого потоку і перевіряти зміни за допомогою утиліти у process explorer.